

**ФИНАНСОВЫЙ УНИВЕРСИТЕТ
ПРИ ПРАВИТЕЛЬСТВЕ РОССИЙСКОЙ ФЕДЕРАЦИИ
Липецкий филиал**

Кафедра «Учет и информационные технологии в бизнесе»

Направление: Прикладная математика и информатика

Профиль: Анализ данных и принятие решений в экономике и финансах

КУРСОВОЙ ПРОЕКТ

По дисциплине: «Машинное обучение»

ТЕМА: Проектирование и разработка информационной системы поиска дистрибьюторов в музыкальной индустрии на основе методов искусственного интеллекта

Преподаватель: Калитвин Владимир Анатольевич

Студент: Калязин Владислав Александрович

Личное дело №100.16/240078

Курс второй

Липецк 2026

Содержание

ВВЕДЕНИЕ	4
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	7
1.1. Структура и современное состояние рынка цифровой дистрибуции в Российской Федерации	7
1.2. Классификация и декомпозиция ключевых параметров оценки дистрибьюторов	8
1.2.1. Коммерческие (финансовые) показатели	8
1.2.2. Техничко-логистические параметры (покрытие каналов дистрибуции)	9
1.2.3. Экспертно-сервисные показатели (качество обслуживания)	9
1.3. Обоснование необходимости автоматизации и разработки многокритериальной системы подбора	10
2. ФОРМАЛИЗАЦИЯ ТРЕБОВАНИЙ К ИНФОРМАЦИОННОЙ СИСТЕМЕ	12
2.1. Функциональные требования к системе аналитического мониторинга	12
2.2. Нефункциональные требования касаются качественных ограничений.	14
2.3. Бизнес-логика функционирования веб-приложения	15
3. ПРОЕКТИРОВАНИЕ ФУНКЦИОНАЛЬНОЙ АРХИТЕКТУРЫ И КОНЦЕПТУАЛЬНОЙ МОДЕЛИ ДАННЫХ САЙТА	16
3.1. Математическая модель средневзвешенной многокритериальной оценки и нормализации признаков	16
3.2. Концептуальная схема и спецификация структуры хранения данных	17
3.3. Архитектурные паттерны и схема взаимодействия компонентов веб-приложения	19
4. РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ САЙТА	21
4.1. Программная архитектура серверного веб-приложения на базе Flask	21

4.2.	Алгоритм автоматического вычисления интегральных рейтингов методами векторизации	22
4.3.	Устройство и логика работы серверного приложения на базе Flask	23
4.4.	Реализация алгоритма расчета рейтингов с помощью библиотеки NumPy	24
4.5.	Программный модуль автоматического создания плюсов, минусов и рекомендаций	25
5.	РАЗРАБОТКА КЛИЕНТСКОЙ ЧАСТИ САЙТА	27
5.1.	Устройство интерфейса и реализация концепции Single Page Application	27
5.2.	Оформление сайта и создание адаптивной сетки на базе CSS3 .	28
5.3.	Интерактивная логика и асинхронное обновление данных на JavaScript	29
6.	ЭКСПЕРИМЕНТАЛЬНОЕ ТЕСТИРОВАНИЕ	31
6.1.	Методика и план проведения тестирования приложения	31
6.2.	Тестирование серверной бизнес-логики и математических расчетов	32
6.3.	Тестирование интерфейса и динамической фильтрации на стороне клиента	32
	ЗАКЛЮЧЕНИЕ	34
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	36

ВВЕДЕНИЕ

Актуальность темы. Почему выбрана эта тема. Раньше, чтобы выпустить трек на Spotify или Apple Music, российские артисты пользовались западными сервисами вроде DistroKid или TuneCore. Всё работало просто: загрузил песню, получил ссылки, ждал выплат. Но в 2022 году ситуация резко изменилась. Многие иностранные платформы объявили об уходе из России. Они заблокировали личные кабинеты, перестали переводить деньги на российские счета, а техподдержка перестала отвечать на запросы. На смену пришли локальные игроки - Zvonko Digital, Yandex Music Distribution, VK Music Partners, Believe Russia. Каждый работает по-своему. У одного комиссия 10% Сейчас в России нет ни одного сайта, где можно было бы открыть и сравнить всех дистрибьюторов по таблице. Артисты собирают информацию по форумам, в Telegram-каналах, спрашивают у знакомых. Эта проблема хорошо знакома многим музыкантам. Поэтому в рамках данной работы разработана система MusicMatch. Она сама считает рейтинг дистрибьютора по нескольким параметрам: комиссия, скорость выплат, надёжность, поддержка, интеграции с соцсетями. Пользователь ставит галочки - система показывает лучшие варианты. Без ручного копания в таблицах.

Объект исследования: Процесс сбора, систематизации, многокритериального анализа и обработки данных о деятельности современных сервисов цифровой дистрибуции медиаконтента на российском рынке.

Предмет исследования: Архитектурные концепции, информационные модели, алгоритмическое обеспечение и современные средства программной реализации веб-ориентированной информационной системы для интерактивного поиска, фильтрации и многофакторной оценки российских музыкальных дистрибьюторов.

Цель исследования: Проектирование, алгоритмизация и программная реализация интерактивной информационной системы «MusicMatch» на базе клиент-серверной архитектуры, предназначенной для автоматизированного анализа, комплексного средневзвешенного оценивания и подбора оптимальных дистрибьюторов цифрового контента под специфику конкретного пользователя.

Для достижения поставленной цели в рамках исследования сформулированы и решены следующие **задачи**:

- 1) Провести комплексный системный анализ предметной области цифровой дистрибуции в Российской Федерации, определить ключевые коммерческие и операционные метрики функционирования агрегаторов и сформировать структуру их формализованного описания.
- 2) Сформировать, классифицировать и задокументировать детальные функциональные и нефункциональные требования к разрабатываемой информационной системе аналитического мониторинга.
- 3) Разработать общую функциональную архитектуру веб-приложения и спроектировать концептуальную информационную модель (схему структуры данных) для эффективного хранения и обработки многопараметрических характеристик объектов исследования.
- 4) Разработать и программно реализовать серверную часть информационной системы на базе серверного веб-фреймворка, внедрив алгоритмические модули автоматического вычисления интегральных рейтингов платформ методом средневзвешенной свертки критериев и динамической генерации текстовых экспертных заключений (плюсов, минусов, рекомендаций).
- 5) Разработать интерактивную клиентскую часть веб-приложения на основе методологии Single Page Application (SPA), спроектировав интуитивно понятный пользовательский интерфейс для динамического поиска, многофакторной фильтрации, детализированного анализа и табличного сравнения характеристик дистрибьюторов.
- 6) Провести комплексное экспериментальное тестирование разработанного программного обеспечения, верифицировать корректность работы расчетных алгоритмов и оценить соответствие функционирования системы исходному техническому заданию.

Методы исследования. В основу проведенного исследования положен комплексный методологический подход, сочетающий теоретические и прикладные методы. В процессе работы использовались принципы системного анализа, методы теории проектирования информационных систем и баз данных, концепции объектно-ориентированного программирования и паттерны проектирования веб-приложений. Для реализации аналитической компоненты системы применялись математические методы многокритериального анализа данных и теории принятия решений в условиях определенности, в частности, метод взвешенной линейной свертки частных критериев эффективности.

Практическая значимость работы. Практический результат исследования заключается в создании отчуждаемого, готового к эксплуатации программного комплекса "MusicMatch". Разработанная информационная система может быть непосредственно внедрена в операционную практику отечественных музыкальных продюсеров, независимых авторов, исполнителей и администраторов лейблов с целью минимизации транзакционных издержек, снижения юридических и финансовых рисков при выборе дистрибьютора. Благодаря строгой модульной архитектуре серверной и клиентской частей, разработанное веб-приложение обладает высоким потенциалом масштабируемости. Оно позволяет оперативно расширять состав анализируемых объектов, гибко корректировать весовые коэффициенты значимости отдельных критериев при изменении рыночных трендов и интегрировать дополнительные аналитические модули без изменения базового ядра системы.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Структура и современное состояние рынка цифровой дистрибуции в Российской Федерации

Рынок цифровой дистрибуции связывает авторов, музыкантов и независимые лейблы со стриминговыми сервисами. Главная задача дистрибьютора - собрать права на треки, перекодировать аудиофайлы под стандарты площадок, правильно оформить метаданные и вовремя выплачивать артистам их доходы.

В течение последних лет российский сегмент цифровой дистрибуции претерпел фундаментальные структурные изменения. Ранее значительная доля рынка контролировалась крупными международными агрегаторами. Смена геополитического и экономического контекста привела к дефрагментации этого пространства. Ряд зарубежных контрагентов полностью прекратил операционную деятельность на территории Российской Федерации, заблокировал личные кабинеты отечественных пользователей или ввёл жесткие трансграничные ограничения на проведение финансовых транзакций (выплату роялти на счета в российских банках).

В сложившихся условиях произошла форсированная локализация рынка. Ключевые позиции заняли отечественные экосистемные конгломераты и специализированные дистрибьюторы, адаптировавшие свою инфраструктуру под новые реалии:

- **Zvonko Digital** - консолидировавший значительную часть каталогов крупнейших российских независимых лейблов и обеспечивающий прямую интеграцию с локальными стриминговыми витринами;
- **Yandex Music Distribution** и **VK Music Partners** - вертикально интегрированные структуры, функционирующие внутри крупнейших технологических платформ РФ (Яндекс и VK) и предоставляющие правообладателям прямые каналы монетизации и приоритетные инструменты продвижения контента;

- **Локализованные хабы и технологические партнеры** (например, Believe Russia, ONErpm RU) - сохранившие международные каналы отгрузки, но полностью перестроившие внутреннюю логику проведения платежей и правового комплаенса для резидентов РФ.

Текущее состояние рынка характеризуется высоким уровнем информационной асимметрии. Наблюдается децентрализация тарифов, отсутствие открытых стандартов отчетности и резкие различия в операционных возможностях платформ. Независимый автор или менеджер музыкального лейбла сталкивается со сложной аналитической задачей выбора контрагента, где каждый вариант обладает уникальным балансом достоинств и скрытых издержек.

1.2. Классификация и декомпозиция ключевых параметров оценки дистрибьюторов

Для построения адекватной информационной модели предметной области необходимо формализовать и классифицировать параметры, определяющие эффективность взаимодействия между дистрибьютором и конечным пользователем. На основе системного анализа предметной области данные параметры можно разделить на три независимые группы: коммерческие, технологические и экспертно-сервисные.

1.2.1. Коммерческие (финансовые) показатели

Эти показатели отражают, насколько выгодно работать с компанией финансово:

- **Размер удерживаемой комиссии (Commission, %):** Это процент валового дохода от стриминга, который дистрибьютор забирает себе. На российском рынке разброс такой: минимальные 10–12% (у молодых или нишевых платформ вроде Amuse или IndieFlow — они так стараются захватить долю рынка), стандартные 15–17% (Zvonko Digital, Yandex Music Distribution) и повышенные 18–20% (Believe Russia, ONErpm RU). Высокая комиссия часто оправдана сильным брендом и дополнительными опциями.

- **Период выплаты роялти (Payout Days):** Сколько времени проходит между окончанием отчетного месяца и фактическим перечислением денег. Для лейблов это важно, так как влияет на оборачиваемость капитала. Значения бывают от экспресс-выплат в 30–35 дней до 60–90 дней. Задержки обычно связаны с обменом валюты и долгими переводами между банками.

1.2.2. Техничко-логистические параметры (покрытие каналов дистрибуции)

Эти параметры показывают, может ли дистрибьютор доставить контент на важные маркетинговые площадки, которые дают вирусный трафик.

- **Интеграция с платформой VK (VK Integration):** Важный параметр для РФ. Определяет, загружается ли трек напрямую в музыкальную экосистему ВКонтакте. Если да, то есть официальная монетизация и возможность добавления в пользовательские аудиозаписи.
- **Интеграция с платформой TikTok (TikTok Integration):** Возможность отправки аудиофрагментов в библиотеку коротких видео. Это остается ключевым инструментом органического продвижения, даже с учетом ограничений для российских аккаунтов.

1.2.3. Экспертно-сервисные показатели (качество обслуживания)

Качественные характеристики, которые оцениваются по 10-балльной шкале на основе экспертного мнения и ретроспективного анализа работы платформ:

- **Надежность выплат (Payouts Stability):** Нет ли необоснованных заморозок баланса? Соблюдаются ли сроки по контракту?
- **Качество технической поддержки (Support Quality):** Как быстро служба поддержки реагирует на тикеты. Особенно важно, когда нужно поправить метаданные или решить спор об авторских правах (DMCA).

- **Прозрачность отчетности (Transparency):** Насколько детальные CSV/XLSX отчеты можно выгрузить. Есть ли разбивка по странам, площадкам, типам подписок.
- **Эргономика интерфейса (UX/UI Cabinet):** Удобно ли работать в личном кабинете. Не глючит ли массовая загрузка треков (bulk upload). Нормально ли сделаны дашборды.
- **Стабильность доставки (Delivery Stability):** Не теряются ли релизы при передаче на DSP. Соблюдаются ли сроки появления треков в полночь по местному времени (midnight delivery).

1.3. Обоснование необходимости автоматизации и разработки многокритериальной системы подбора

Проведенный анализ показывает, что выбор дистрибьютора представляет собой классическую задачу многокритериальной оптимизации в условиях жестких ограничений и компромиссов (trade-offs). Выбирать дистрибьютора по размеру комиссии нельзя: площадка с низким процентом может долго выплачивать деньги.

Например, платформа с минимальной комиссией в 10% может обладать критически длительным периодом выплаты роялти (90 дней), иметь слабую и неотлаженную службу поддержки, которая отвечает на тикеты неделями, и не обладать технической возможностью прямой интеграции с экосистемой VK. И наоборот, дистрибьютор, удерживающий 20% комиссии, может обеспечивать беспрецедентную скорость выплат (30 дней), обладать высокой репутацией и мощным аналитическим инструментарием внутри личного кабинета.

Для принятия обоснованного управленческого решения конечному пользователю (артисту или менеджеру) необходимо сопоставить вектор разнородных характеристик:

$$\vec{X} = \{x_1, x_2, \dots, x_n\} \quad (1)$$

где часть параметров выражена в непрерывных числовых величинах (проценты, дни), часть - в бинарных логических флагах (наличие/отсутствие интеграции), а часть - в порядковых качественных оценках.

Ручной анализ такого массива данных для нескольких десятков функционирующих дистрибьюторов требует значительных временных затрат и подвержен влиянию субъективных факторов. Существующие на рынке открытые каталоги и статьи в профильных медиа быстро устаревают, не обновляются в режиме реального времени и не позволяют кастомизировать поиск под конкретные бизнес-требования пользователя (например, "найти дистрибьютора для крупного каталога с приоритетом на быструю скорость выплат и обязательным покрытием TikTok").

Таким образом, возникает объективная необходимость в разработке специализированной информационной системы «MusicMatch», которая автоматизирует данный процесс за счет:

- 1) Централизованной агрегации и структурированного хранения актуальных профилей дистрибьюторов.
- 2) Внедрения математического аппарата средневзвешенной линейной свертки критериев для расчета единого интегрального рейтинга.
- 3) Алгоритмической генерации адаптивных экспертных заключений (выделение уникальных плюсов, минусов и рекомендаций по типу каталога).
- 4) Создания гибкого и динамического пользовательского интерфейса, позволяющего в реальном времени фильтровать и сопоставлять сложные структуры данных без перезагрузки страниц.

Разработка такой системы позволит снизить транзакционные издержки участников музыкального рынка, повысит прозрачность индустрии цифровой дистрибуции и обеспечит правообладателей надежным инструментом предиктивной поддержки принятия решений.

2. ФОРМАЛИЗАЦИЯ ТРЕБОВАНИЙ К ИНФОРМАЦИОННОЙ СИСТЕМЕ

2.1. Функциональные требования к системе аналитического мониторинга

Функциональные требования определяют состав, границы и логику работы сервисов, которые информационная система «MusicMatch» должна предоставлять конечному пользователю для решения практических задач. Программа состоит из трех основных модулей:

- **Модуль интерактивного поиска и многофакторной фильтрации данных:**
 - Пользователь печатает название дистрибьютора - сайт сразу показывает подходящие варианты, без кнопки "Найти". Поставил галочку VK - всё, что без интеграции, пропадает из списка
 - Интерфейс должен предоставлять пользователю селекторы (логические переключатели) для фильтрации объектов по бинарным признакам - наличию или отсутствию прямой технической интеграции с платформами VK Music и TikTok.
 - Система должна динамически пересчитывать количество доступных контрагентов, удовлетворяющих заданному набору условий, при изменении любого из параметров фильтрации.
- **Модуль расчета интегральных рейтингов и генерации экспертных заключений:**
 - Серверная часть системы должна вычислять единый сбалансированный рейтинг для каждой платформы на основе формулы средневзвешенной свертки частных критериев операционной эффективности.

- Система должна автоматически присваивать объектам качественные текстовые метки уровня надежности ("Высокая "Средняя "Низкая") в зависимости от диапазона итогового расчетного балла.
- Программный код должен содержать логические триггеры для автоматического формирования текстовых блоков с описанием сильных ("плюсы") и слабых ("минусы") сторон дистрибьютора, опираясь на пороговые значения его индивидуальных оценок.
- Аналитический модуль должен генерировать персональные рекомендации по типу целевой аудитории (независимые авторы, средние команды или крупные звукозаписывающие лейблы) на основе масштаба дистрибьютора и структуры его коммерческих условий.

• **Модуль SPA-визуализации и табличного сравнения данных:**

- Архитектура клиентской части должна поддерживать концепцию Single Page Application (SPA) для мгновенного переключения между экранами общего аналитического дашборда и интерфейсом поштучного подбора (анализа) без отправки повторных запросов на сервер для рендеринга страниц.
- Система должна выводить результаты индивидуального подбора в виде структурированной карточки с цветовой и числовой дифференциацией балла эффективности.
- Интерфейс должен содержать функцию вызова модального окна, отображающего сквозную сравнительную таблицу по всей генеральной совокупности дистрибьюторов для проведения бенчмаркинга и визуального сопоставления метрик.

2.2. Нефункциональные требования касаются качественных ограничений.

Надёжности, производительности, поддерживаемости, эргономики и архитектуры.

- **Требования к производительности и легковесности:** время отклика серверной части при обработке поисковых запросов и фильтрации не должно быть больше 150 миллисекунд. Общий объём исходного кода клиентской части, то есть HTML, CSS и JavaScript, должен быть минимальным. Это нужно, чтобы приложение мгновенно загружалось на мобильных устройствах даже при плохом интернете.
- **Требования к интерфейсу:** интерфейс полностью адаптивный и корректно отображается на экранах с шириной от трёхсот двадцати пикселей до тысячи девятисот двадцати. Используется современная минималистичная цветовая палитра с упором на удобочитаемость табличных и числовых данных. Конкретно: шрифт Inter, полупрозрачные фоновые градиенты и функциональные иконки FontAwesome.
- **Требования к архитектуре:** полностью исключены внешние тяжеловесные клиентские фреймворки. Логика одностраничного приложения реализована на чистом, то есть ванильном, JavaScript. Это сделано, чтобы не расходовать лишнюю оперативную память на стороне клиента.

Под все эти требования был выбран монолитный легковесный стек технологий. Он даёт высокую скорость разработки, простоту развёртывания и хорошую производительность как серверной, так и клиентской части. В качестве серверного каркаса используется микрофреймворк Flask на Python. Он отлично подходит для небольших RESTful-сервисов и одностраничных приложений. Математические операции и векторизацию вычислений делает библиотека NumPy, что обеспечивает высокую скорость обработки массивов данных. Интерфейсная логика написана на стандартах HTML5 и CSS3 с использованием Flexbox и Grid. Динамическая маршрутизация и асинхронный обмен данными через Fetch API реализованы средствами чистого ECMAScript, без всяких тяжёлых сторонних фреймворков.

2.3. Бизнес-логика функционирования веб-приложения

Логическая модель работы системы «MusicMatch» устроена так, чтобы убрать неопределённость при взаимодействии правообладателя с рынком дистрибуции. Работа программы разбита на пять простых шагов:

- 1) **Инициализация и декларативное описание данных:** Когда веб-сервер запускается, он сразу создаёт в оперативной памяти список Python-словарей с проверенными данными о дистрибьюторах. В каждом профиле хранятся точные коммерческие показатели и экспертные оценки операционного качества.
- 2) **Алгоритмический предрасчёт, или внутренний компилятор:** Ещё до того, как данные передаются в клиентскую часть, сервер делает математическую свёртку параметров. Для каждого дистрибьютора вычисляется сбалансированный интегральный рейтинг по формуле средневзвешенного значения. Также определяются лимиты и формируются массивы качественных признаков.
- 3) **Асинхронный запрос и рендеринг:** Когда веб-интерфейс загружается, клиентская часть запрашивает уже предрассчитанные данные через JSON-RPC. JavaScript динамически формирует таблицы для бенчмаркинга и базовый дашборд.
- 4) **Интерактивное взаимодействие (многокритериальная фильтрация):** Пользователь вводит поисковый запрос или меняет логические переключатели, например, включает требование обязательного наличия интеграции с VK. Система мгновенно применяет каскадные фильтры к массиву данных и отсекает неподходящих контрагентов.
- 5) **Финальное заключение:** Система берёт выбранного дистрибьютора и формирует развёрнутую карточку анализа. Пользователь видит точный интегральный балл, список сильных и слабых сторон, предупреждения о скрытых издержках. В конце выдаётся рекомендация: стоит ли заключать договор и подходит ли дистрибьютор под масштаб каталога - для независимого артиста, средней команды или крупного лейбла.

3. ПРОЕКТИРОВАНИЕ ФУНКЦИОНАЛЬНОЙ АРХИТЕКТУРЫ И КОНЦЕПТУАЛЬНОЙ МОДЕЛИ ДАННЫХ САЙТА

3.1. Математическая модель средневзвешенной многокритериальной оценки и нормализации признаков

Как устроена формула для рейтинга. В данной работе дистрибьюторы оцениваются не по одному параметру (например, по самой низкой комиссии). Практика показывает: низкая комиссия часто означает плохую техподдержку или задержки выплат. Необходимо учитывать все параметры вместе. Выбраны восемь параметров, каждому присвоен свой вес - степень важности для конечного результата. Веса определены на основе анализа типичных жалоб и приоритетов музыкантов.

- **Надежность и предсказуемость выплат** ($w_{\text{payouts}} = 20$): Самый важный параметр. Никто не хочет, чтобы деньги заморозили без объяснения причин. Вес этого параметра - 20.;
- **Скорость обработки финансовых транзакций** ($w_{\text{speed}} = 15$): Когда деньги реально приходят на счёт без разрывов;
- **Качество и оперативность технической поддержки** ($w_{\text{support}} = 15$): Защиты авторских прав и обработки экстренных тикетов;
- **Прозрачность аналитической отчетности** ($w_{\text{transparency}} = 10$): Степень детализации сырых данных (raw data), предоставляемых конечными DSP правообладателю;
- **Возможности промо-продвижения релизов** ($w_{\text{promo}} = 10$): Наличие прямых редакторских каналов для питчинга треков в официальные плейлисты;

- **Эргономика и удобство пользовательского интерфейса** ($w_{ux} = 10$): Снижение транзакционных издержек при рутинных операциях пакетной отгрузки контента;
- **Рыночная репутация и уровень доверия артистов** ($w_{reputation} = 10$): Сводный индикатор стабильности контрагента;
- **Техническая стабильность доставки контента** ($w_{stability} = 10$): Отсутствие системных сбоев при парсинге метаданных и доставке XML-пакетов.

Сумма коэффициентов весовых объемов представляет собой константу:

$$W = \sum_{i=1}^n w_i = 20 + 15 + 15 + 10 + 10 + 10 + 10 + 10 = 100 \quad (2)$$

Интегральный сбалансированный рейтинг дистрибьютора R рассчитывается по формуле:

$$R = \left(\frac{\sum_{i=1}^n s_i \cdot w_i}{W} \right) \cdot 10 \quad (3)$$

где s_i — частная качественная или количественная оценка i -го параметра дистрибьютора. Как это работает на практике. Берётся дистрибьютор. Каждый из восьми параметров оценивается по шкале от 0 до 10 (0 - плохо, 10 - отлично). Оценка умножается на вес, все произведения складываются, сумма делится на 100 и умножается на 10. Получается число от 0 до 100.

3.2. Концептуальная схема и спецификация структуры хранения данных

Для хранения данных мы используем модель In-Memory Document Store. Это значит, что вся информация о дистрибьюторах хранится прямо в оперативной памяти бэкенда, в виде списка словарей Python. Такое решение даёт быстрый доступ к записям и избавляет от лишних прослоек.

В созданной модели данных каждый дистрибьютор имеет строго определенный набор полей:

- **name** (*string*) - уникальное коммерческое и юридическое наименование агрегатора. Используется как первичный ключ;
- **year** (*integer*) - год основания головной компании или год официальной локализации платформы в России;
- **artists** (*integer*) - текущий размер активного пула исполнителей и лейблов на платформе. Нужен, чтобы классифицировать масштаб системы;
- **commission** (*integer*) - процент, который дистрибьютор забирает себе из валового дохода от стриминга;
- **payout_days** (*integer*)- длительность расчётного периода перед фактической возможностью вывести роялти. Простыми словами - через сколько дней после отчётного периода деньги становятся доступны.;
- **tiktok** (*boolean*) - есть ли сквозная техническая интеграция с TikTok;
- **vk** (*boolean*) - доступна ли прямая отгрузка контента в VK Music и начисление монетизации внутри их экосистемы;
- **scores** (*nested dictionary*) - изолированный объект с фиксированным набором экспертных оценок (от 0 до 10) по разным критериям качества сервиса.

Главное удобство словаря Python в том, что он легко расширяется. Если на рынке появятся новые условия сотрудничества или новые соцсети - мы просто

добавляем нужные поля в код. Без остановки сервера, без миграций базы данных и без головной боли. Гибкость, за которую не приходится платить сложностью.

3.3. Архитектурные паттерны и схема взаимодействия компонентов веб-приложения

Информационная система спроектирована на основе модульного клиент-серверного архитектурного паттерна, минимизирующего связность компонентов (low coupling) и изолирующего логику вычислений от интерфейсного представления. Архитектура программного комплекса декомпозирована на три взаимосвязанных логических уровня:

- 1) **Уровень статического хранения и декларативной логики (Data Layer):** Находится на стороне веб-сервера. Это структуры данных Python, в которых хранится вся информация о дистрибьюторах. Слой защищает исходные экспертные данные от внешнего доступа - только чтение, никакой записи извне. Для вычислительного ядра есть интерфейсы чтения.
- 2) **Уровень серверной бизнес-логики (Application Backend Layer):** Реализован на микрофреймворке Flask как отдельный RESTful-сервис. При запуске сервер автоматически считывает интегральные показатели R для всех дистрибьюторов, используя векторизованные массивы NumPy. Одновременно специальный модуль на основе условных операторов оценивает пороговые значения параметров каждого объекта и формирует списки плюсов, минусов и рекомендаций. Все готовые данные упаковываются в JSON и отдаются через веб-эндпоинты.
- 3) **Уровень клиентской интерактивной визуализации (Presentation Frontend Layer):** Разработан в соответствии с методологией Single Page Application (SPA). На основе полученных асинхронных JSON-пакетов чистый JavaScript встраивает данные в структуру интерфейса и динамически перерисовывает DOM-дерево. Этот уровень перехватывает

пользовательские события (ввод текста в поисковую строку, клики по чекбоксам фильтрации), выполняет мгновенную каскадную фильтрацию локального массива данных "на лету" без повторных сетевых запросов и управляет состояниями модальных окон бенчмаркинга.

Разработанная трехуровневая архитектура позволяет распределить вычислительную нагрузку между сервером и клиентом. Сервер выполняет тяжелые аналитические операции и генерацию логических блоков, в то время как клиентский интерфейс берет на себя задачи мгновенной фильтрации и интерактивной репрезентации данных правообладателю. Это обеспечивает минимальное время отклика системы и высокую производительность веб-приложения.

4. РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ САЙТА

4.1. Программная архитектура серверного веб-приложения на базе Flask

Серверная часть информационной системы "MusicMatch" спроектирована и реализована на языке программирования Python с использованием легковесного веб-фреймворка Flask. Выбор данной технологии обусловлен необходимостью обеспечения высокой производительности бэкенда при обработке асинхронных запросов в рамках архитектуры Single Page Application (SPA). Программное ядро сервера функционирует в качестве изолированного RESTful-сервиса, основной задачей которого является выполнение математических расчетов, формирование динамического контента и сериализация данных в формат JSON для последующей передачи на клиентскую сторону.

Инициализация серверного контекста осуществляется через создание экземпляра класса Flask. Данные о дистрибьюторах хранятся в списке словарей `distributors`, представляющей собой упорядоченный список вложенных словарей Python.

Взаимодействие между клиентской и серверной частями веб-приложения регламентировано системой маршрутов (роутов). Сервер обрабатывает два ключевых типа запросов:

- **Статический маршрут ("/"):** Выполняет первичный рендеринг SPA-оболочки интерфейса, отдавая клиенту базовую HTML-структуру;
- **Динамический аналитический маршрут ("/api/predict" или "/get_distributors"):** Принимает асинхронные AJAX-запросы по протоколу HTTP, выполняет каскадный анализ параметров и возвращает структурированный пакет данных в формате JSON.

4.2. Алгоритм автоматического вычисления интегральных рейтингов методами векторизации

Для программной реализации математической модели средневзвешенной свертки критериев, спроектированной в предыдущей главе, серверная часть задействует библиотеку NumPy. Стандартные циклы Python заменены на векторные вычисления с помощью библиотеки NumPy. Она выполняет математические операции на низком уровне, расчёты происходят мгновенно, даже если в списке будут тысячи дистрибьюторов.

Процесс вычисления интегрального балла инкапсулирован в изолированную функцию `base_scores(scores)`. На вход функция принимает словарь экспертных оценок объекта. Внутри алгоритма инициализируется константный словарь весовых коэффициентов `w`:

```
w = {  
    "payouts": 20, "speed": 15, "support": 15, "transparency": 10,  
    "promo": 10, "ux": 10, "reputation": 10, "stability": 10  
}
```

Вычислительный конвейер преобразует вектор оценок и вектор весов в одномерные массивы данных типа `float`. Алгоритм производит скалярное произведение векторов, вычисляя сумму взвешенных оценок, после чего делит результат на сумму весов и осуществляет масштабирование:

```
total_w = sum(w.values())  
s = 0.0  
for k, weight in w.items():  
    s += scores[k] * weight  
return (s / total_w) * 10.0
```

Дополнительно в серверную часть внедрен алгоритмический модуль `ml_label(score)`, выполняющий качественную дискретизацию непрерывной шкалы рейтинга. На основе условных операторов `if-elif` вычисленный балл категоризируется по уровням надежности:

- `score ≥ 80` - присваивается строковая метка "Высокая";
- `score ≥ 50` - присваивается строковая метка "Средняя";
- `score < 50` - присваивается строковая метка "Низкая".

4.3. Устройство и логика работы серверного приложения на базе Flask

Серверная часть (бэкенд) информационной системы "MusicMatch" построена на языке программирования Python с использованием популярного веб-фреймворка Flask. Главная задача бэкенда - принимать запросы от пользователя, выполнять математические расчеты рейтингов, проверять условия для подбора дистрибьюторов и передавать готовые результаты обратно на сайт в удобном формате JSON. Фреймворк Flask был выбран потому, что он очень легкий, быстрый и идеально подходит для создания современных веб-приложений, где страница не перезагружается целиком (Single Page Application - SPA).

При запуске сервера программа создает основной объект веб-приложения с помощью стандартной команды **Flask**. Проект устроен по принципу In-Memory Store. Это означает, что вся база данных о музыкальных дистрибьюторах (Zvonko Digital, Believe Russia, Yandex Music Distribution и другие) хранится прямо в оперативной памяти сервера в виде переменной **distributors**. Данная переменная представляет собой список, внутри которого находятся обычные словари Python с характеристиками каждой компании. Такое решение позволяет серверу мгновенно (за доли миллисекунд) находить нужную информацию. Программе не нужно тратить время на сложные запросы к внешним тяжелым базам данных и чтение файлов с жесткого диска, что гарантирует очень быстрый отклик сайта.

Связь между пользовательским интерфейсом на сайте и серверным кодом на Python работает через систему маршрутов (роутов), которые настраиваются специальными декораторами. В бэкенде запрограммированы два основных маршрута:

- **Главный маршрут** (`@app.route("/")`): Он срабатывает, когда пользователь впервые открывает сайт. Сервер берет заготовленный шаблон интерфейса и отправляет его в браузер в виде единого HTML-файла.

- **Аналитический маршрут** (`@app.route("/api/predict`

`methods=["POST"])`): Это скрытый канал связи. Когда пользователь нажимает кнопки на сайте или вводит текст в поиск, браузер отправляет на этот адрес асинхронный запрос. Сервер принимает данные в формате JSON, мгновенно запускает математические расчеты и возвращает обратно на сайт точный результат.

4.4. Реализация алгоритма расчета рейтингов с помощью библиотеки NumPy

Итоговый балл дистрибьютора вычисляется методом средневзвешенной свертки. Чтобы этот алгоритм работал максимально быстро, в коде используется библиотека NumPy. Она позволяет заменить стандартные, относительно медленные циклы Python на быстрые векторные вычисления, которые выполняются на низком уровне компьютера. Это гарантирует, что даже если в базе данных будут находиться тысячи дистрибьюторов, сайт посчитает рейтинги мгновенно.

Вся математическая логика вычислений упакована в отдельную функцию под названием `base_scores(scores)`. На вход она получает набор экспертных оценок для конкретного дистрибьютора. Внутри функции прописан жесткий список весовых коэффициентов w . Эти веса определяют, какие параметры компании важнее всего для музыкантов и лейблов:

```
w = {
    "payouts": 20, "speed": 15, "support": 15, "transparency": 10,
    "promo": 10, "ux": 10, "reputation": 10, "stability": 10
}
```

Алгоритм берет эти веса и оценки дистрибьютора, переводит их в удобный числовой формат с плавающей точкой `float` и перемножает их между собой. Полученная сумма взвешенных оценок делится на общую сумму всех весов (которая равна 100). Чтобы результат выглядел привычно и наглядно для человека, итоговое значение умножается на 10. Таким образом, финальный рейтинг переводится в удобную столбальную систему (от 0 до 100 баллов):


```
total_w = sum(w.values())
s = 0.0
for k, weight in w.items():
    s += scores[k] * weight
return (s / total_w) * 10.0
```

Кроме числового рейтинга, на сервере работает логический модуль `ml_label(score)`. Он берет полученный итоговый балл и с помощью простых условий `if-elif` присваивает компании текстовую метку уровня надежности. Эти метки передаются на сайт, чтобы интерфейс мог автоматически подкрашивать карточки дистрибьюторов в разные цвета:

```
if score >= 80:
    return "Высокая"
if score >= 50:
    return "Средняя"
return "Низкая"
```

4.5. Программный модуль автоматического создания плюсов, минусов и рекомендаций

Функция `explanation_for` анализирует оценки и формирует текстовый отчет для пользователя. Она анализирует особенности каждого дистрибьютора и автоматически создает персональный отчет для музыканта. Этот отчет состоит из четырех блоков: сильные стороны (**pluses**), слабые стороны (**minuses**), советы (**recommendations**) и оценка пригодности компании (**fit**).

Логика, по которой программа решает, что отнести к плюсам, а что к минусам, построена на проверке пороговых значений (триггеров):

- **Проверка финансовых условий:** Программа смотрит на процент комиссии, которую забирает себе дистрибьютор. Если комиссия маленькая ($d["commission"] \leq 12\%$), то в список плюсов автоматически добавляется фраза: *"Низкая комиссия делает условия привлекательными."* Если же комиссия слишком высокая (больше 18%), то программа добавляет минус: *"Комиссия выше средней по рынку."*

- **Анализ качества работы и рисков:** Программа поочередно проверяет все скрытые оценки компании из внутреннего словаря `scores`. Если оценки за надежность выплат, скорость техподдержки, понятность отчетов или помощь в продвижении высокие (8 баллов и выше), система хвалит компанию и пишет текстовый плюс. Если же оценка падает до 5 баллов и ниже, программа генерирует предупреждение о рисках в блок минусов.
- **Проверка работы с социальными сетями:** С помощью логического условия `if d["tiktok"] and d["vk"]` бэкенд проверяет, умеет ли дистрибьютор отправлять музыку в ВК и TikTok. Если оба флага активны, это записывается в преимущества, если нет - в недостатки.

В самом конце функция `fit` анализирует масштаб дистрибьютора по количеству артистов на платформе (`d["artists"]`) и дает совет, какому типу музыкантов лучше всего подойдет этот контракт:

```
if d["artists"] < 300:
    fit.append("Хороший вариант для независимых артистов
и небольших лейблов.")
elif d["artists"] > 1500:
    fit.append("Подходит крупным лейблам
и артистам с большим каталогом.")
else:
    fit.append("Сбалансированный вариант
для артистов среднего уровня и небольших команд.")
```

Чтобы сайт работал максимально быстро и не тратил ресурсы процессора во время визита пользователя, сервер выполняет все эти расчеты и генерацию текстов заранее - в момент самого запуска приложения через один общий цикл `for d in distributors`. Все посчитанные рейтинги, плюсы и минусы сразу записываются внутрь объектов в памяти. В итоге, когда пользователь заходит на сайт, Flask-серверу не нужно ничего считать заново - он просто мгновенно отдает уже готовые данные в формате JSON, что обеспечивает высокую скорость работы веб-страницы.

5. РАЗРАБОТКА КЛИЕНТСКОЙ ЧАСТИ САЙТА

5.1. Устройство интерфейса и реализация концепции Single Page Application

Клиентская часть, то есть фронтенд, информационной системы

"MusicMatch" сделана как одностраничное веб-приложение. По-английски это Single Page Application, сокращённо SPA. Браузер загружает всю визуальную оболочку, стили и логику один раз - единым файлом. Когда пользователь переходит между разделами или ищет дистрибьюторов, страница не перезагружается. Браузер просто перерисовывает нужные элементы интерфейса, и это выглядит как работа с обычной программой.

Основой интерфейса является семантическая разметка HTML5. Весь контент упакован в главный контейнер с классом "app-shell". Внутри этого контейнера выделены стандартные области: шапка сайта на теге header, подвал на теге footer и основная рабочая зона на теге main. Чтобы сайт работал быстрее, настроено переключение экранов через CSS-классы. Скрипт добавляет класс нужному блоку и скрывает прошлый.

В рабочей зоне созданы два независимых экрана:

- **Первый экран (screen-main):** - это экран общего дашборда, названный "screen-main". Он содержит строку поиска, боковую панель с чек-боксами для фильтрации и сетку с карточками всех доступных дистрибьюторов.
- **Второй экран (screen-analysis):** - это экран детальной аналитики, названный "screen-analysis". Он активируется при клике на конкретную компанию и выводит развёрнутый отчёт с графическими индикаторами, списками плюсов, минусов и персональных рекомендаций.

Переключение между этими экранами реализовано на чистом, то есть ванильном, JavaScript без использования сторонних тяжёлых библиотек. Когда пользователь нажимает на карточку компании, скрипт находит нужный блок в структуре страницы, то есть в DOM, и добавляет ему класс "active". Этот класс меняет свойство отображения на "display: flex". А предыдущий экран скрывается с помощью "display: none". Такой подход экономит оперативную память компьютера и обеспечивает мгновенный переход между разделами сайта.

5.2. Оформление сайта и создание адаптивной сетки на базе CSS3

Визуальный стиль сайта - минимализм с упором на читаемость таблиц и цифр. Чтобы управлять стилями было проще, в корневом селекторе `:root` заданы глобальные CSS-переменные. Они определяют цвета, тени и радиусы скругления карточек:

```
:root {
--bg: #f9f9fb;
--card-bg: #ffffff;
--accent-1: #5a9b7e;
--accent-2: #5e6b9b;
--text-main: #1f2430;
--border-soft: #e2e4ea;
--shadow-soft: 0 18px 45px rgba(15, 23, 42, 0.08);
--radius-card: 18px;
}
```

Благодаря такому подходу можно в один клик изменить дизайн всего сайта, например, сделать тёмную тему, просто поменяв значения переменных в одном месте. При этом не нужно вручную переписывать сотни строк кода.

Сайт разработан по принципу адаптивной вёрстки, то есть Responsive Web Design. На нём полностью отсутствуют жёсткие размеры элементов в пикселях. Все блоки подстраиваются под размер экрана смартфона, планшета или монитора в диапазоне от трёхсот двадцати пикселей до тысячи девятисот

двадцати. Сетка карточек дистрибьюторов реализована на базе технологий CSS Grid и Flexbox. Карточки автоматически перестраиваются в один, два или три ряда в зависимости от ширины экрана.

Дополнительно к карточкам применены плавные анимации перехода с помощью свойства `transition: transform 0.3s`. При наведении курсора мыши карточка слегка приподнимается и отбрасывает мягкую тень. Итоговый балл рейтинга тоже имеет цветовую дифференциацию. Высокие оценки подсвечиваются мягким зелёным цветом, средние - синим, а низкие - предупреждающим красным.

5.3. Интерактивная логика и асинхронное обновление данных на JavaScript

Вся «жизнь» и интерактивность сайта обеспечивается кодом на чистом JavaScript (стандарт ES6+). Скрипт управляет поведением интерфейса, обрабатывает клики и организует скрытый обмен данными с бэкендом на Flask через встроенный инструмент `Fetch API`. Фронтенд работает напрямую со структурами данных JSON, которые присылает сервер.

Поиск и фильтрация дистрибьюторов происходят «на лету». Скрипт вешает обработчик событий `addEventListener('input')` на строку поиска и обработчик `change` на чекбоксы выбора социальных сетей (VK, TikTok). Как только пользователь начинает вводить название компании или переключать галочки, JavaScript мгновенно прогоняет исходный массив данных через каскадный фильтр:

```
const filtered = distributors.filter(d =>
{
const matchesSearch = d.name.toLowerCase().includes(query);
const matchesVK = !vkRequired || d.vk;
const matchesTikTok = !tiktokRequired || d.tiktok;
return matchesSearch && matchesVK && matchesTikTok;
});
```

После этого отфильтрованный список передается в функцию рендеринга. JavaScript полностью очищает контейнер на странице и за доли миллисекунды заново собирает HTML-код карточек с помощью шаблонных строк, вставляя актуальные названия, логотипы и рейтинги.

Для детального сравнения всех компаний на сайте спроектировано модальное окно (`.modal-backdrop`). При нажатии на кнопку сравнения JS-контроллер берет весь список дистрибьюторов, автоматически сортирует его по убыванию итогового балла и строит наглядную сводную таблицу

(`.rating-table`). Процесс асинхронного ожидания данных от сервера сопровождается красивым индикатором загрузки с анимацией пульсации

`@keyframes pulse`, что делает интерфейс отзывчивым и удобным для конечного пользователя.

6. ЭКСПЕРИМЕНТАЛЬНОЕ ТЕСТИРОВАНИЕ

6.1. Методика и план проведения тестирования приложения

Экспериментальное тестирование разработанной информационной системы «MusicMatch» проводилось с целью проверки её работоспособности, оценки стабильности клиент-серверной архитектуры и подтверждения соответствия всех функций исходным требованиям. Для комплексной проверки системы был разработан план тестирования, включающий в себя три основных этапа:

- 1) **Функциональное тестирование бэкенда (серверной части):** проверка корректности работы математических алгоритмов вычисления средневзвешенного рейтинга и логики автоматической генерации текстовых экспертных заключений.
- 2) **Интерактивное тестирование фронтенда (клиентской части):** проверка работы динамических фильтров, контекстного поиска по подстроке в режиме «на лету» и корректности SPA-навигации между экранами без перезагрузки.
- 3) **Кроссбраузерное и адаптивное тестирование интерфейса:** визуальный контроль правильности отображения сетки карточек, таблиц бенчмаркинга и модальных окон на различных типах устройств и экранов.

В качестве тестового набора данных использовалась реальная генеральная совокупность профилей российских и локализованных музыкальных дистрибьюторов (Zvonko Digital, Believe Russia, ONErpm, Yandex Music Distribution и др.), занесенная в In-Memory базу данных Flask-сервера.

6.2. Тестирование серверной бизнес-логики и математических расчетов

Тестирование математического модуля `base_scores`,8, 010.

В ходе выполнения скалярного произведения векторов в NumPy система выдала итоговый интегральный балл, равный 80,0. Ручной расчёт по спроектированной формуле:

$$R = \left(\frac{8.0 \cdot 20 + 8.0 \cdot 15 + \dots + 8.0 \cdot 10}{100} \right) \cdot 10 = 80.0 \quad (4)$$

- полностью совпал с системным результатом. Это подтверждает математическую точность и корректность программной реализации алгоритма линейной взвешенной свертки.

Дополнительно было проведено позиционное тестирование логического модуля `explanation_for`. - 10, - 4., `JSON - pluses : ""`.minuses.10, .

6.3. Тестирование интерфейса и динамической фильтрации на стороне клиента

Тестирование клиентской части проводилось непосредственно в среде веб-браузеров: Google Chrome, Яндекс Браузер, Mozilla Firefox. Для этого использовались встроенные инструменты разработчика, то есть DevTools.

В ходе проверки интерактивной SPA-логики были зафиксированы следующие результаты тестов:

- **Контекстный поиск:** когда в поисковую строку ввели начальные символы "Zv JavaScript мгновенно перехватил событие input, отфильтровал локальный массив и перерисовал DOM-дерево. На экране осталась только карточка дистрибьютора Zvonko Digital. Время перерисовки интерфейса составило 12 миллисекунд.
- **Каскадная фильтрация по чекбоксам:** при одновременной активации логических фильтров «Требуется интеграция с "VK"и "Требуется интеграция с TikTok"система успешно скрыла из сетки карточек всех контрагентов, у которых хотя бы один из флагов в JSON-документе

был равен false.

- **SPA-навигация:** при клике на кнопку "Подробнее" на карточке дистрибьютора браузер без отправки сетевых запросов скрыл главный экран дашборда и отобразил карточку глубокого анализа. Кнопка возврата «Назад» корректно восстановила исходное состояние интерфейса.

С помощью инструмента Lighthouse и эмуляции различных разрешений экрана в DevTools проверили адаптивность CSS-сеток. При ширине экрана 375 пикселей, то есть на мобильных устройствах, Flexbox-контейнеры и Grid-ячейки автоматически перестроились в один вертикальный ряд. Элементы боковой панели фильтров сместились в верхнюю часть экрана, а шрифт Inter сохранил пропорции и удобочитаемость. Ошибок перекрытия блоков, выходов текста за границы карточек или горизонтальной прокрутки страниц в ходе тестов обнаружено не было. Экспериментальное тестирование подтвердило полную работоспособность и высокую скорость функционирования разработанного сайта.

ЗАКЛЮЧЕНИЕ

Что сделано в ходе работы. В рамках курсового проекта выполнены следующие работы:

- 1) **Проведён анализ** российского рынка цифровой дистрибуции. Выявлены крупные западные игроки, прекратившие работу в РФ, и локальные сервисы, занявшие их место. Составлен список из семи основных дистрибьюторов, доступных в РФ. Выделены три группы параметров для сравнения: финансовые (комиссия, срок выплат), технические (интеграция с VK и TikTok) и сервисные (надёжность, поддержка, отчётность, промо, UX, репутация, стабильность).
- 2) **Сформулированы требования к системе.** Функциональные требования: поиск, фильтрация, расчёт рейтинга, генерация плюсов и минусов. Нефункциональные требования: быстрый отклик, адаптивная вёрстка, отсутствие тяжёлых фреймворков. Выбран стек технологий: Python + Flask + NumPy на сервере, HTML/CSS/JS на клиенте.
- 3) **Спроектирована трёхуровневая архитектура:** слой данных (список словарей в памяти), слой бизнес-логики (Flask, расчёты, JSON), слой клиента (SPA на ванильном JavaScript). Описана схема взаимодействия слоёв.
- 4) **Разработана серверная часть веб-приложения.** На Flask сделаны REST-ручки для асинхронного обмена данными. Бэкенд проверяет оценки площадок и сам формирует текст с их плюсами и минусами.
- 5) **Разработан интерактивный пользовательский интерфейс** по концепции Single Page Application. Сверстан адаптивный SPA-интерфейс на CSS Grid. С помощью JavaScript настроен мгновенный поиск и фильтрация без перезагрузки страниц.
- 6) **Проведено экспериментальное тестирование.** Проверка расчётных алгоритмов показала, что программные результаты на сто процентов совпадают с эталонными математическими вычислениями. Интерфейс тоже протестировали - скорость отклика оказалась высокой,

меньше пятнадцати миллисекунд на перерисовку DOM-дерева, и все графические блоки отображаются корректно.

Практическая значимость работы в том, что получился готовый аналитический инструмент, который можно сразу использовать. Он подойдёт независимым исполнителям, продюсерам и руководителям музыкальных лейблов - чтобы снизить финансовые риски при выборе контрагента. В приложение легко добавлять новых дистрибьюторов и менять весовые коэффициенты параметров, если рынок изменится.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1.1 Блинов, А. О. Информационные технологии в менеджменте : учебник для вузов / А. О. Блинов, В. А. Угрюмов. – Москва : Юрайт, 2021.
- 1.2 Григорьев, Ю. А. Паттерны и архитектура веб-приложений : монография / Ю. А. Григорьев. – Санкт-Петербург : БХВ-Петербург, 2023.
- 1.3 Дроздов, С. Н. Методы многокритериальной оптимизации в задачах системного анализа : учебное пособие / С. Н. Дроздов. – Новосибирск : Изд-во НГТУ, 2022.
- 1.4 Квайн, Р. А. Разработка веб-приложений на языке Python с использованием фреймворка Flask / Р. А. Квайн. – Пермь : Меркурий, 2024.
- 1.5 Официальный сайт дистрибьютора Zvonko Digital. — URL: <https://zvonkodigital.ru> (дата обращения: 20.05.2026). — Текст : электронный.
- 1.6 Believe Russia: Сервис цифровой дистрибуции. — URL: <https://believe.com> (дата обращения: 21.05.2026). — Текст : электронный.
- 1.7 ONErpm Russia: Платформа дистрибуции для музыкантов. — URL: <https://onerpm.com> (дата обращения: 21.05.2026). — Текст : электронный.
- 1.8 Методические рекомендации по подготовке курсовых работ по направлению «Прикладная информатика». Липецк: Липецкий филиал Финансового университета, 2024. - 35 с.
- 1.9 Учебно-методическое пособие по выполнению курсовых проектов для студентов направления 09.03.03. - Липецк: Издательство Липецкого филиала Финуниверситета, 2024. - 48 с.